Chapter 6

APIs and contracts, REST, Java Frameworks Spring and Spring Boot

Reference:

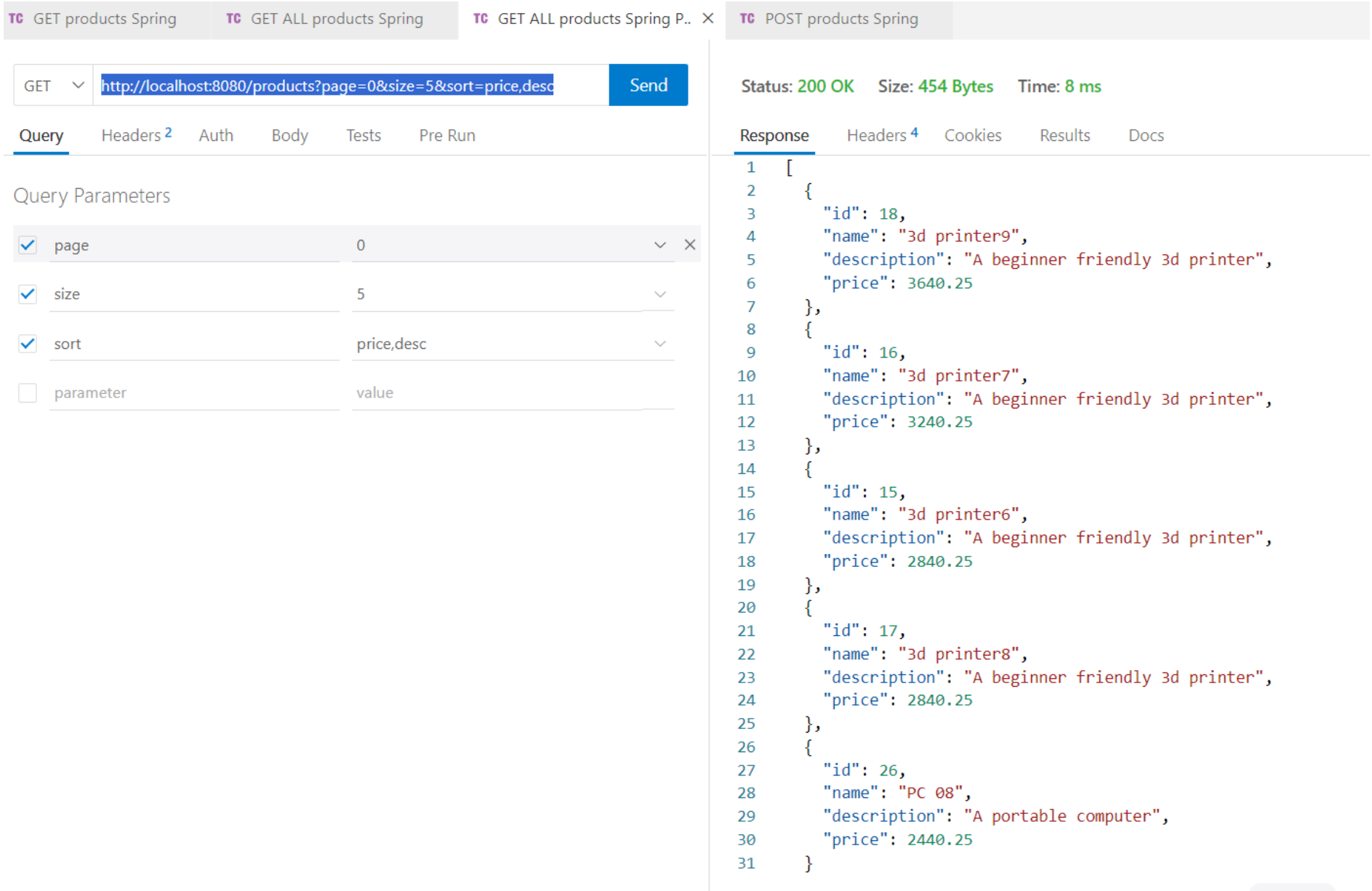
https://spring.academy/courses/building-a-rest-api-with-spring-boot/lessons/spring-boot

Outline

- Java Framework Spring and Spring boot
- API contracts and JSON
- REST API and HTTP CRUD operations
- REST API design principles and best practices
- MVC architecture and the implementation of HTTP operations handlers (controllers)
- Implementation options for views and models

Objective of creating a server

Use a communication protocol like HTTP to communicate with the server to retrieve or create data for example



Spring

- Spring is a comprehensive framework that provides various modules for building different types of applications. It's like having a giant toolbox with every tool you could possibly need to build anything you want:
  - **Spring MVC** is used to create web applications with the MVC design pattern,
  - **Spring Data** for data access,
  - **Spring Security** for authentication, authorization and protection.
- This versatility comes at a cost. Setting up a Spring application requires a lot of configuration, and developers need to manually configure various components of the framework to get an application up and running.

Spring Boot

- Fortunately, Spring Boot makes working with Spring much simpler.
- Spring Boot is like a more opinionated version of Spring. It comes with many **pre-configured** settings and dependencies that are commonly used in Spring applications.
- Makes it really easy to **get started quickly**, without having to worry about setting up everything from scratch.
- Plus, Spring Boot comes with **an embedded web server,** so you can easily create and deploy web applications without needing an external server.
- In summary, Spring is a powerful, comprehensive framework that gives you a lot of flexibility, but can be a bit overwhelming to set up. Spring Boot is a more opinionated, streamlined version of Spring that comes with a lot of **built-in features** to help you get started quickly and easily.

Inversion of Control

- **Inversion of Control (IoC)** is a design principle where the control flow of a program is handed over from your custom code to an external component (usually a framework or container). Instead of your code calling library routines to perform generic tasks, the framework calls your code at the appropriate time, the control is inverted
- **Dependency Injection (DI):** Objects receive their dependencies from an external injector/container rather than creating them themselves. This is the most common IoC pattern.
- **Event-driven callbacks:** Frameworks call your handlers when events occur (GUI frameworks, web frameworks).

Simple Example of IoC

```
// Service interface
public interface GreetingService {
    String greet();
}

// Implementation 1
public class SelemService implements GreetingService {
    public String greet() { return "Selem"; }
}
```



```
// Implementation 2
public class HelloService implements GreetingService {
    public String greet() { return "Hello"; }
}

// Consumer that does not create its dependency
public class Greeter {
    private final GreetingService service;
    public Greeter(GreetingService service) { this.service = service; }
    public void say() { System.out.println(service.greet()); }
}

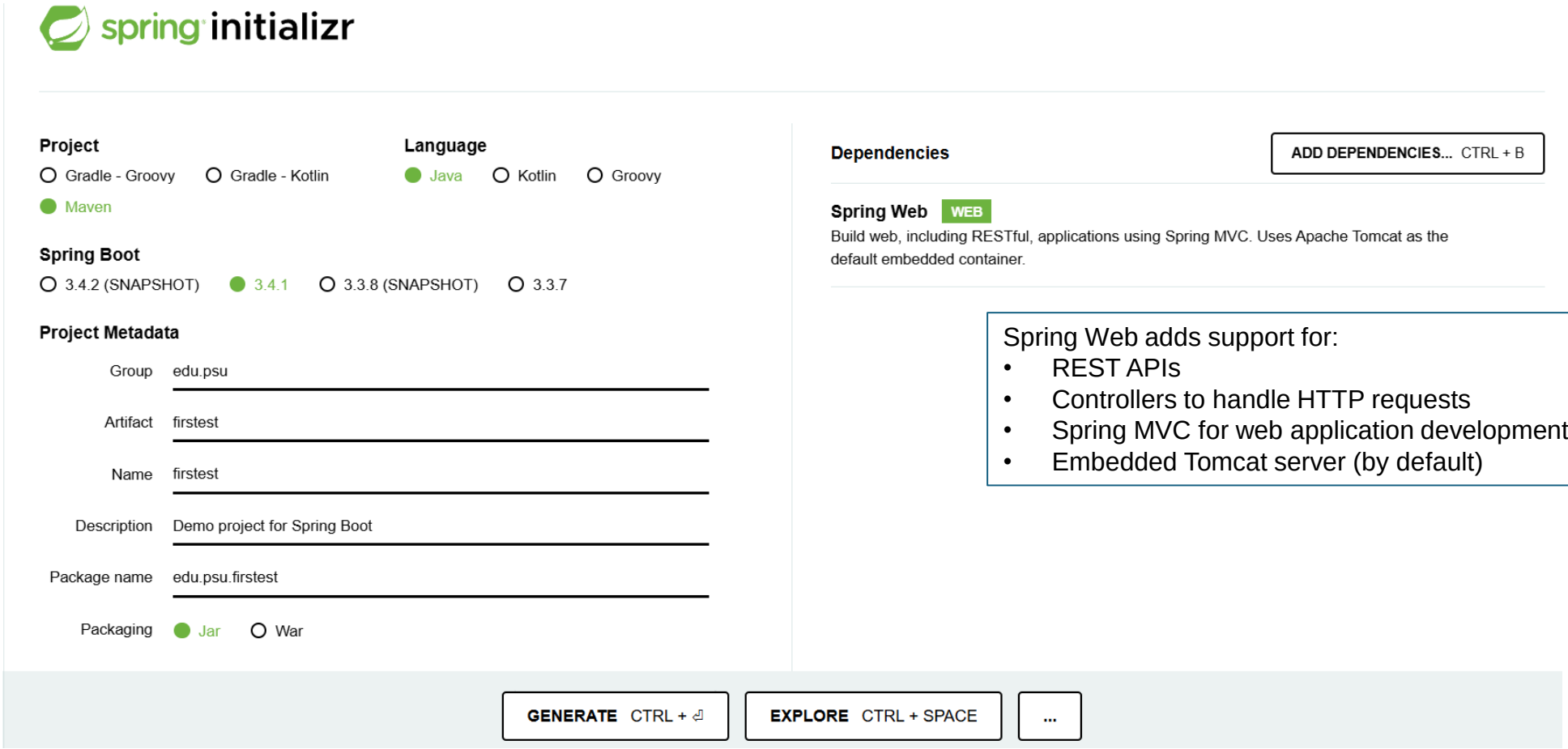
// In an IoC setup, a container would construct SelemService or HelloService and inject it into Greeter.
```

Spring’s Inversion of control container

- Spring Boot takes advantage of Spring Core's **Inversion of Control** (IoC) container.
- Spring Boot allows you to configure how and when dependencies are provided to your application at runtime. This puts you in control of how your application operates in different scenarios.
- For example, you might want to use a different database for local development than for your live, public-facing application. Your application code shouldn't care about this distinction; if it did, you'd have to **hard-code** every possible scenario into your application logic. Instead, Spring Boot allows you to provide an **external configuration** that specifies how and when such dependencies are used.
- In Java, Inversion of control is mostly implemented through **dependency injection** (DI) where dependencies are "injected" into applications at runtime.

Spring Initializr

- Spring Initializr is an online service that is used to create a Spring boot application with the dependencies we need.



Spring Web adds support for:
- REST APIs
- Controllers to handle HTTP requests
- Spring MVC for web application development
- Embedded Tomcat server (by default)

Web API Contracts and JSON

We're developing a web API. This brings up a lot of questions about how the API should behave:

- How should API consumers interact with the API?
- What data do consumers need to send in various scenarios?
- What data should the API return to consumers, and when?
- What does the API communicate when it's used incorrectly (or something goes wrong)?

Whenever possible, API providers and consumers should discuss these scenarios and come to agreements.

Better yet, they should **document** these agreements not only in a shared documentation system, but also in a way that supports automated tests passing (or failing) based upon these decisions.

This is where the concept of contracts comes in.



API Contract

- We define an API contract as a formal agreement between a software provider and a consumer that abstractly communicates how to interact with each other.
- This contract defines how API providers and consumers interact, what data exchanges look like, and how to communicate success and failure cases.
- The provider and consumers do not have to share the same programming language, only the same API contracts. Below is an example of a first API contract.

```
Request
  URI: /cashcards/{id}
  HTTP Verb: GET
  Body: None

Response:
  HTTP Status:
    200 OK if the user is authorized and the Cash Card was successfully retrieved
    401 UNAUTHORIZED if the user is unauthenticated or unauthorized
    404 NOT FOUND if the user is authenticated and authorized but the Cash Card cannot be found
Response Body Type: JSON
Example Response Body:
  {
    "id": 99,
    "amount": 123.45
  }
```

What is JSON?

- JSON (Javascript Object Notation) provides a data interchange format that represents the particular information of an object in a format that you can easily read and understand.
- Other popular data formats include XML (Extensible Markup Language). When compared to XML, JSON reads and writes quicker, is easier to use, and takes up less space. You can use JSON with most modern programming languages and on all major platforms. It also works seamlessly with JavaScript-based applications.
- For these reasons, JSON has largely superseded XML as the most widely used format for APIs used by Web apps, including REST APIs.

```
{
   "id": 99,
   "amount": 123.45
}
```

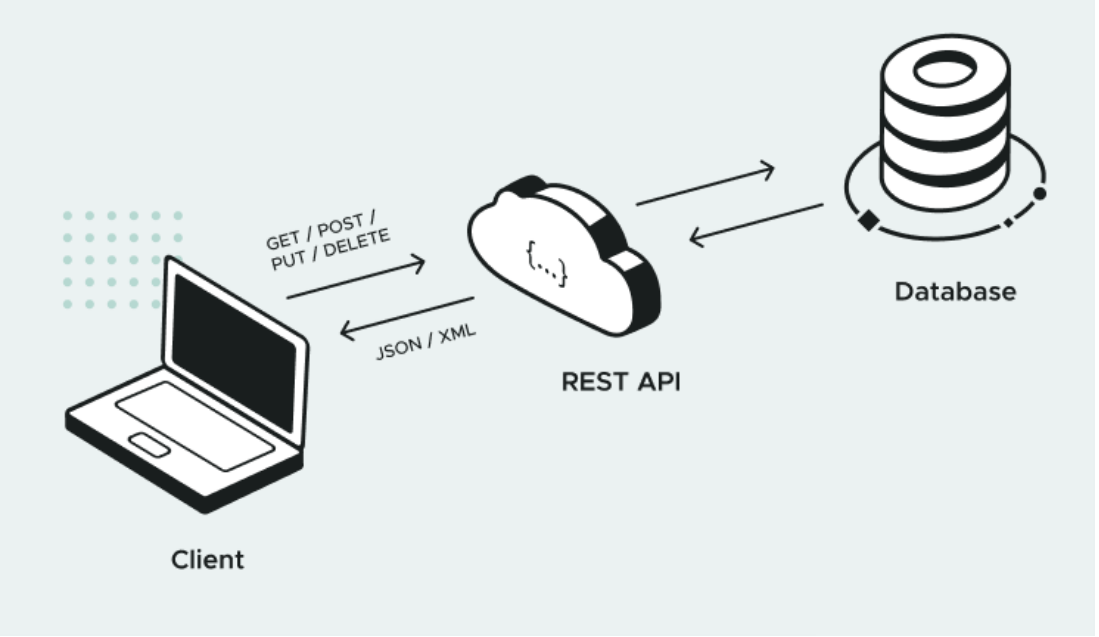
Spring Boot automatically converts Java objects to JSON. Spring Boot makes this very easy:

What is REST, CRUD, and HTTP?

- HTTP (Hypertext Transfer Protocol) is the communication protocol that allows a client, such as a browser or mobile app, to send requests to a server and receive responses.
- Let's start with a concise definition of **REST**: Representational* State Transfer. In a RESTful system, data objects have Resource Representations (in JSON for example). The purpose of a RESTful API (Application Programming Interface) is to manage the state of these Resources.

  - You can think of "state" as being "value" and "Resource Representation" being an "object serialized as a string". Therefore, REST is just a way to manage how the values of things are represented (for transfer). Those things might be accessed via an API, and are often stored in a persistent data store, such as a database.

* See next slide for an example



Example of a single resource representation

Here, we will use JSON for resource's state representations (state here means data values of the resource) :

```
{
      "<resource name in singular>" :
            {
                  "<property 1>" : <value 1>,
                  "<property 2>" : <value 2>,
                  ...
                  "<property n>" : <value n>,
                  "<reference property 1>" : <value 1>,
                  "<reference property 2>" : <value 2>,
                  ...
                  "<reference property n>" : <value n>,
            }
}
```

Example of a single resource representation


```
{
    "name": "France",
    "capital": "Paris",
    "population": 67364357,
    "area": 551695,
    "currency": "Euro",
    "languages": ["French"],
    "region": "Europe",
    "subregion": "Western Europe",
    "flag":
"https://upload.wikimedia.org/wikipedia/commons/c/c3/Flag_of_France.svg"
}
```

Example of a single resource representation

```
{
  "appConfig": {
    "appName": "MyApp",
    "version": "1.0.0",
    "debugMode": true,
    "supportedLanguages": ["en", "fr", "es"]
  },
  "server": {
    "host": "localhost",
    "port": 8080,
    "useHttps": true,
    "allowedIPs": ["127.0.0.1", "192.168.1.1"]
  },
  "database": {
    "type": "mysql",
    "host": "db.example.com",
    "port": 3306,
    "username": "dbuser",
    "password": "securepassword",
    "databaseName": "myappdb"
  },
  "logging": {
      "level": "info",
      "logToFile": true,
      "logFilePath": "/var/log/myapp.log",
      "logFormat": "json"
    },
    "userPreferences": {
      "theme": "dark",
      "notifications": {
        "email": true,
        "sms": false,
        "push": true
      },
      "privacySettings": {
        "trackLocation": false,
        "shareUsageData": true
      }
    }
}
```

What is REST, CRUD, and HTTP? (2)

- CRUD stands for “Create, Read, Update, and Delete”. These are the four basic operations that can be performed on objects in a data store. REST has specific guidelines for implementing each one.
- Another important concept is the Hypertext Transfer Protocol. In **HTTP**, a caller sends a Request to a URI. A web server receives the request, and routes it to a request handler. The handler creates a Response, which is then sent back to the caller.
- The components of the Request and Response are:
  - Request
    - Method (GET, POST, DELETE…)
    - URI (also called Endpoint)
    - Body
  - Response
    - Status Code
    - Body

For **C**REATE: use HTTP method POST.
For **R**EAD: use HTTP method GET.
For **U**PDATE: use HTTP method PUT.
For **D**ELETE: use HTTP method DELETE.

REST API Endpoints

- Let’s consider that we are creating a REST API to provide CRUD operations for managing “Product” data. The endpoint URI for “Product” objects should begin with **/v1/products** .
- READ, UPDATE, and DELETE operations require we provide the unique identifier of the target resource. For example, to READ, UPDATE, or DELETE a Product with the identifier of “52", the endpoint would be **/v1/products/52**.
- Notice that we do not provide a unique identifier for the CREATE operation, CREATE will have the side effect of creating a new Product with a new unique ID. No identifier should be provided when creating a new Product because the application will create a new unique identifier for us.

Operation	API Endpoint	HTTP Method	Response Status
Create	/v1/products	POST	201 (CREATED)
Read	/v1/products/{id}	GET	200 (OK)
Update	/v1/products/{id}	PUT	204 (NO CONTENT)
Delete	/v1/products/{id}	DELETE	204 (NO CONTENT)

The request body

We need to submit data to the API. This is often referred to as the request body. The CREATE and UPDATE operations require that a request body contains the data needed to properly create or update the resource. For example, a new Product might have an initial price value, and an UPDATE operation might change that value.

Product Example

Let's use the example of a Read endpoint. For the Read operation, the URI (endpoint) path is
/v1/products/{id}, where {id} is replaced by an actual Product identifier without the curly braces, and the HTTP method is GET.
In GET requests, the body is empty. So, the request to read the Product with an id of 123 would be:

Request:

Method: GET
URL: http://product.example.com/v1/products/123
Body: (empty)

The response

The response to a successful Read request has a body containing the JSON representation of the requested Resource, with a Response Status Code of 200 (OK). Therefore, the response to the above Read request would look like this:

Response:

Status Code: 200

Body:

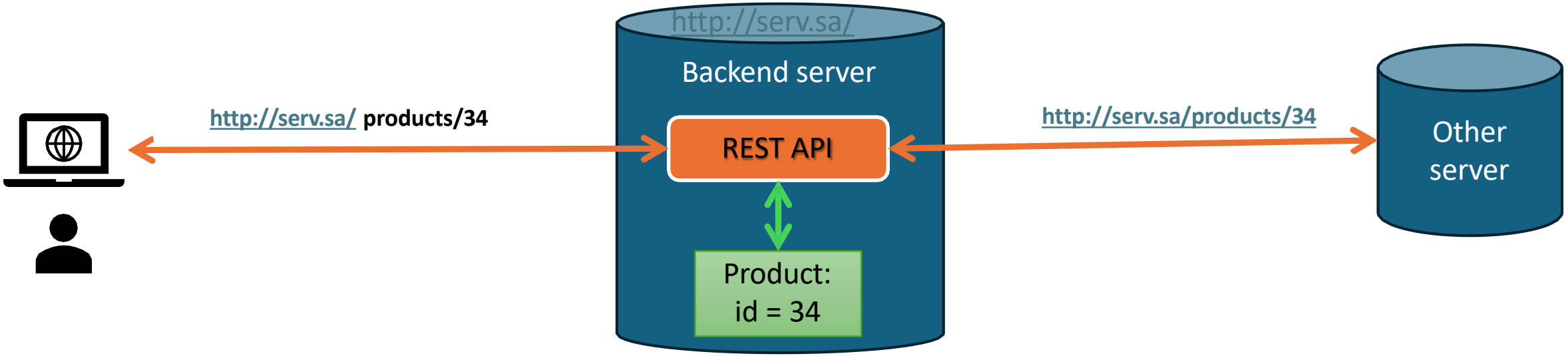
```
{
    "id": 123,
    "amount": 25.00
}
```

REST API principles

- A REST (is an acronym for REpresentational State Transfer) API (also called a RESTful API or RESTful web API) is an application programming interface (API) that conforms to the design principles of the representational state transfer (REST) *architectural style*.
- Developers can develop REST APIs using virtually any programming language and support a variety of data formats. The only requirement is that they align to the six REST design principles, also known as architectural constraints.

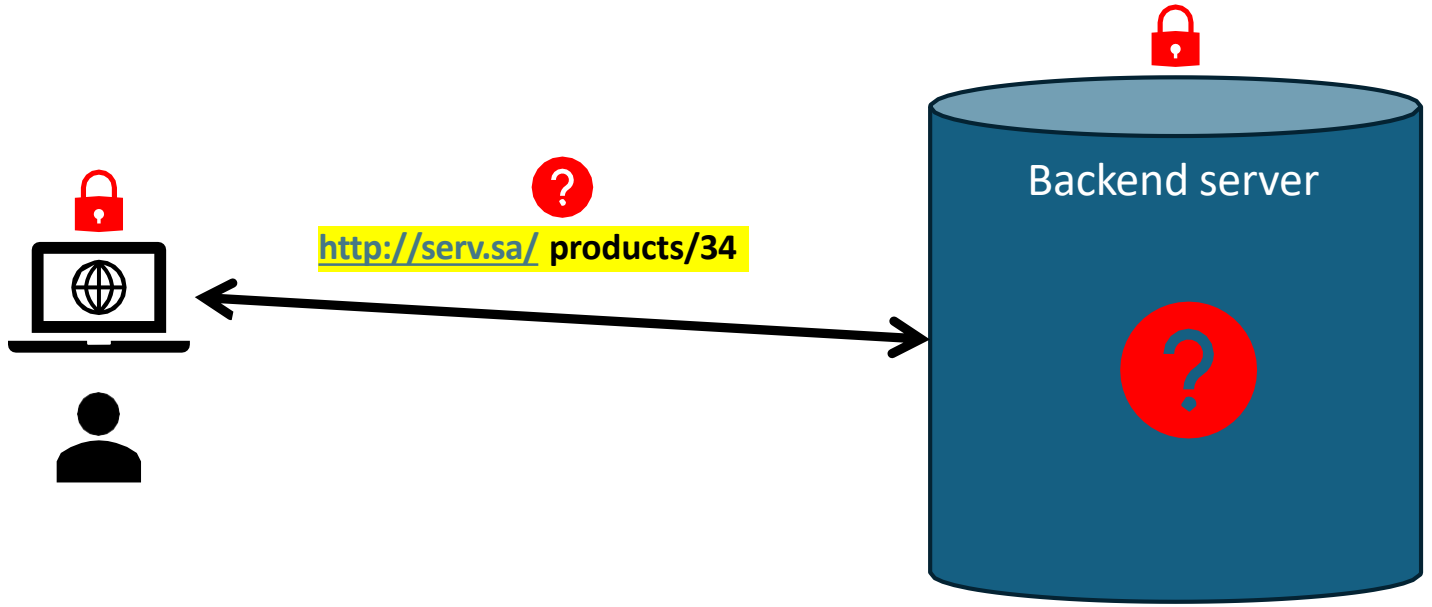
REST API principles: (1) Uniform Interface

- All API requests for the same resource should look the same, no matter where the request comes from (unique URI for every resource).
- The REST API should ensure that the same piece of data, such as the name or email address of a user, belongs to only one uniform resource identifier (URI). Resources should not be too large but should contain every piece of information that the client might need.



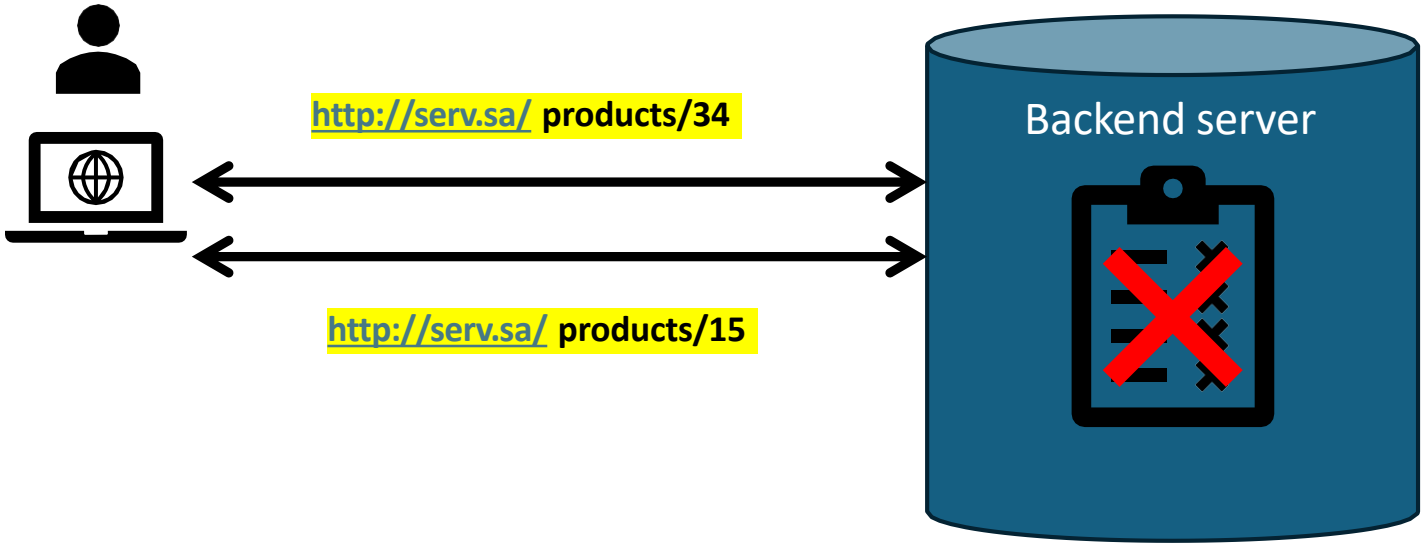
REST API principles: (2) Client-server decoupling

- In REST API design, client and server applications must be completely independent of each other. The only information that the client application should know is the URI of the requested resource i.e. it cannot interact with the server application in any other ways. Similarly, a server application should not modify the client application other than passing it to the requested data via HTTP.



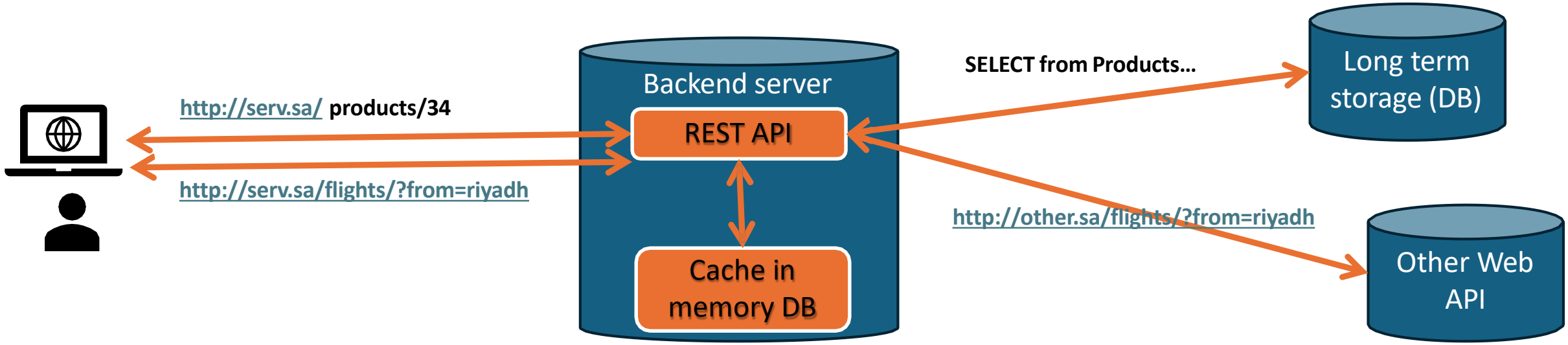
REST API principles: (3) Statelessness

- REST APIs are stateless, which means that each request needs to include all of the information necessary for processing it. In other words, REST APIs do not require any server-side sessions. Server applications are not allowed to store any data related to a client request (except for authentication).



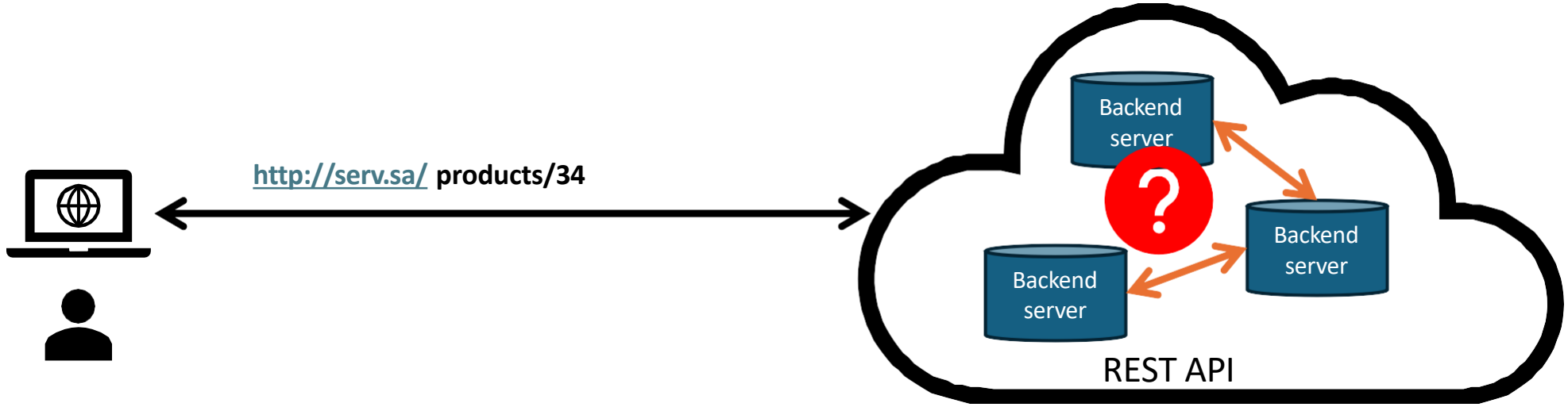
REST API principles: (4) Cacheability

- When possible, resources should be cacheable on the client or server side. Server responses also need to contain information about whether caching is allowed for the delivered resource. The goal is to improve performance on the client side, while increasing scalability on the server side.
- We can add caching to return data from the local memory cache instead of querying the database to get the data every time we want to retrieve some data that users request. The good thing about caching is that users can get data faster. However, the data that users get may be outdated. This may also lead to issues when debugging in production environments when something goes wrong as we keep seeing old data.



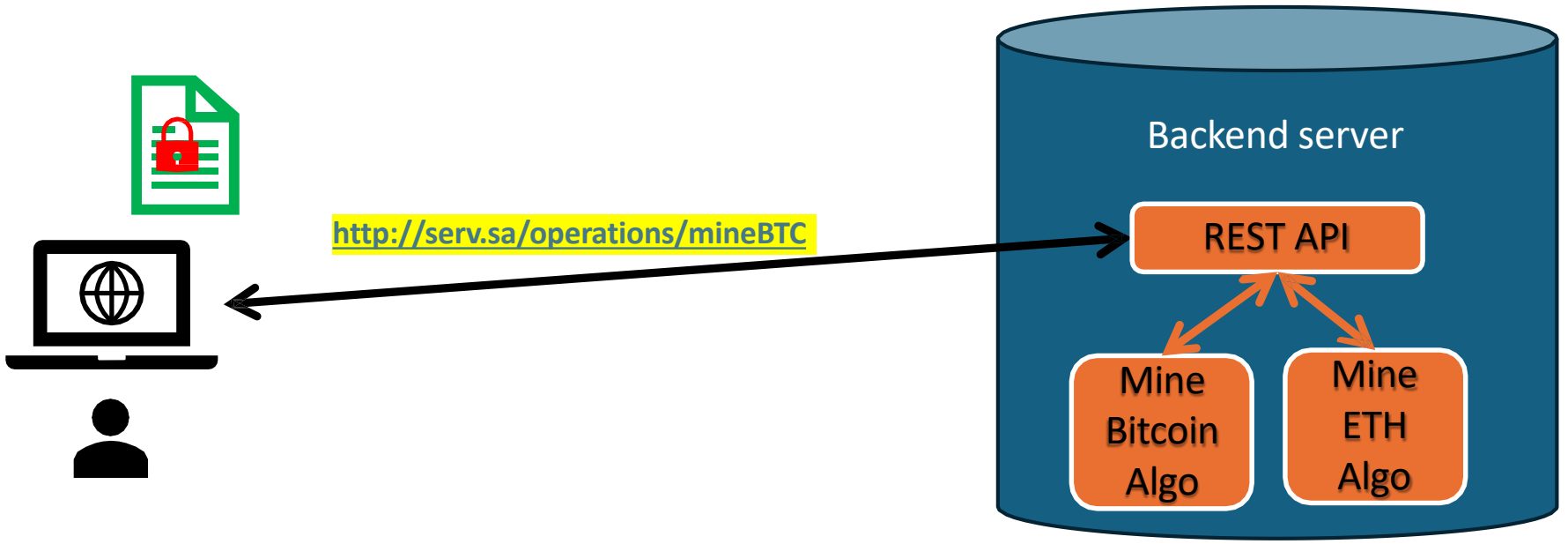
REST API principles: (5) Layered system architecture

- In REST APIs, the calls and responses go through different layers. As a rule of thumb, do not assume that the client, and server applications connect directly to each other. There may be a number of different intermediaries in the communication loop. REST APIs need to be designed so that neither the client nor the server can tell whether it communicates with the end application or an intermediary.



REST API principles: (6)(optional) Code on demand

- REST APIs usually send static resources, but in certain cases, responses can also contain executable code (such as Java applets).
- In these cases, the code should only run on-demand.



REST API best practices: Accept and Respond with JSON

- REST APIs should accept JSON for request payload and also send responses as JSON except when sending files.
- JSON is the standard for transferring data. Almost every networked technology can use it.
  - JavaScript has built-in methods to encode and decode JSON either through the Fetch API or another HTTP client.
  - Server-side technologies have libraries that can decode JSON without doing much work.
- To make sure that when our REST API app responds with JSON that clients interpret it as such, we should set Content-Type in the response header to application/json.

Response

Header:

{

status: 200,

content-type: application/json,

....

REST API best practices: Use nouns instead of verbs in endpoint paths

- We should not use verbs in our endpoint paths. Instead, we should use the nouns which represent the entity that the endpoint that we are retrieving or manipulating as the pathname. This is because our HTTP request method already has the verb (GET, POST, etc.).
- We should create routes like GET /articles/ for getting news articles. Likewise, POST /articles/ is for adding a new article , PUT /articles/:id is for updating the article with the given id. DELETE /articles/:id is for deleting an existing article with the given ID.

GET http://serv.sa/execute/mineBTC

GET http://serv.sa/operations/mineBTC

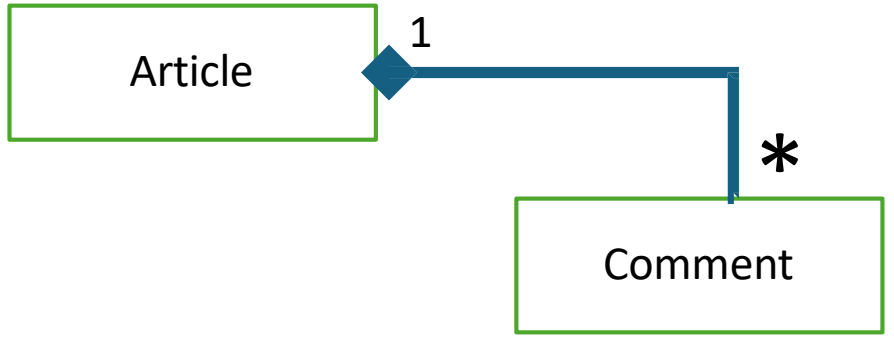
GET http://serv.sa/getArticles/56

GET http://serv.sa/articles/56

REST API best practices: Use Logical nesting on endpoints

- When designing endpoints, it makes sense to group those that contain associated information. That is, if one object can contain another object, you should design the endpoint to reflect that.
- This is good practice regardless of whether your data is structured like this in your database. In fact, it may be advisable to avoid mirroring your database structure in your endpoints to avoid giving attackers unnecessary information.
- For example, if we want an endpoint to get the comments for a news article, we should append the /comments path to the end of the /articles path:

  GET /articles/:articleId/comments



REST API best practices: Handle errors gracefully and return standard error codes

- To eliminate confusion for API users when an error occurs, we should handle errors gracefully and return HTTP response codes that indicate what kind of error occurred.
- Common error HTTP status codes include:

Response Status Code	Usage
400 Bad Request	This means that client-side input fails validation.
401 Unauthorized	User not authorized to access a resource when he isn't authenticated.
403 Forbidden	This means the user is authenticated, but is not allowed to access a resource.
404 Not Found	This indicates that a resource is not found.
500 Internal server error	This is a generic server error.
502 Bad Gateway	This indicates an invalid response from an upstream server.
503 Service Unavailable	This indicates that something unexpected happened on server side (It can be anything like server overload, some parts of the system failed, etc.).

REST API best practices: **Allow filtering, sorting, and pagination**

- The databases behind a REST API can get very large. Sometimes, there's so much data that it shouldn’t be returned all at once because it’s way too slow or will bring down our systems. Therefore, we need ways to filter items.
- We also need ways to paginate data so that we only return a few results at a time. Filtering and pagination both increase performance by reducing the usage of server resources :

/v1/employees?department=marketing&page=3&sort=age

REST API best practice: Versioning

- We may have different versions of API if we're making any changes to them that may break client code.
- This way, we can gradually phase out old endpoints instead of forcing everyone to move to the new API at the same time. The v1 endpoint can stay active for people who don’t want to change (or need more time), while the v2, with its new features, can serve those who are ready to upgrade. This is especially important if our API is public. We should version them so that we won't break third party apps that use our APIs.
- Versioning is usually done with /v1/, /v2/, etc. added at the start of the API path: /v1/employees

Idempotent API: a REST principle

- **Idempotency (ذو مفعول ثابت)** means that the effect of a successfully performed request on a server resource is independent of the number of times it is executed (one or more).
- When we design the REST APIs, we must realize that API consumers can make mistakes. Consumers can write the client code in such a way that there can be duplicate requests coming to the APIs. These duplicate requests may be unintentional as well as intentional (e.g. due to timeout or network issues). We have to make our APIs ***fault-tolerant*** so that duplicate requests do not leave the system unstable.
- Also, idempotency is a key enabler for efficient caching and optimization strategies. Caches (like the Redis database) can store and serve the results of idempotent requests to reduce the load on servers and improve response times.
- If we follow the REST principles in designing our APIs, we will have automatically **idempotent REST APIs** for GET, PUT, DELETE. Only POST and PATCH are not guaranteed to be idempotent.

(HTTP PATCH is used for making partial updates to an existing resource without replacing the entire resource (like incrementing a value)).

Summary about REST API design

Essential features:

- Uniform Interface
- Client-server decoupling
- Statelessness
- Cacheability
- Layered system architecture
- Code on demand

Best practices:

- Accept and Respond with JSON
- Use nouns instead of verbs in endpoint paths
- Logical nesting on endpoints
- Error handling with proper error codes
- filtering, sorting and pagination
- versioning

REST in Spring Boot

Spring Annotations and Component Scan

One of the main things Spring does is to configure, instantiate and inject objects.

These objects are called Spring Beans, and are usually created by Spring (as opposed to using the Java ***new*** keyword).

You can direct Spring to create Beans in several ways. One way is to annotate a class with a Spring Annotation, which directs Spring to create an instance of the class during Spring's Component Scan phase. This happens at application startup. The Bean is stored in Spring's **IoC container**. From here, the bean can be **injected** into any code that requests it.



```
package psu.edu.ch06.tuto01.controller;

import org.springframework.web.bind.annotation.RestController;

@RestController
public class ProductController {

}
```

REST in Spring Boot

- Annotations are used to define beans, inject dependencies and create REST endpoints
- Can be broadly divided into Core Spring Boot annotations and Request Handling and Controller annotations

REST in Spring Boot

Core Spring Boot	Request Handling and Controller
@SpringBootApplication	@Controller
@SpringBootConfiguration	@RestController
@EnableAutoConfiguration	@RequestMapping @GetMapping, @PutMapping, @PostMapping, @PatchMapping, @DeleteMapping
@SpringBootConfiguration	@RequestParam
@EnableAutoConfiguration	@PathVariable
@ComponentScan	@RequestBody
@ConditionalOnClass	@ResponseBody
@ConditionalOnBean	@ModelAttribute
…	…

REST in Spring Boot

- Types of Spring Framework Annotations



Spring Framework Annotations

REST in Spring Boot

```
@SpringBootApplication
public class crud02Application {
 public static void main(String[] args) {
  ConfigurableApplicationContext context = SpringApplication.run(crud02Application.class, args);
        //Arrays.stream(context.getBeanDefinitionNames()).forEach(System.out::println);
       }
}
```

```
package psu.edu.ch06.tuto01.controller;

import org.springframework.web.bind.annotation.RestController;

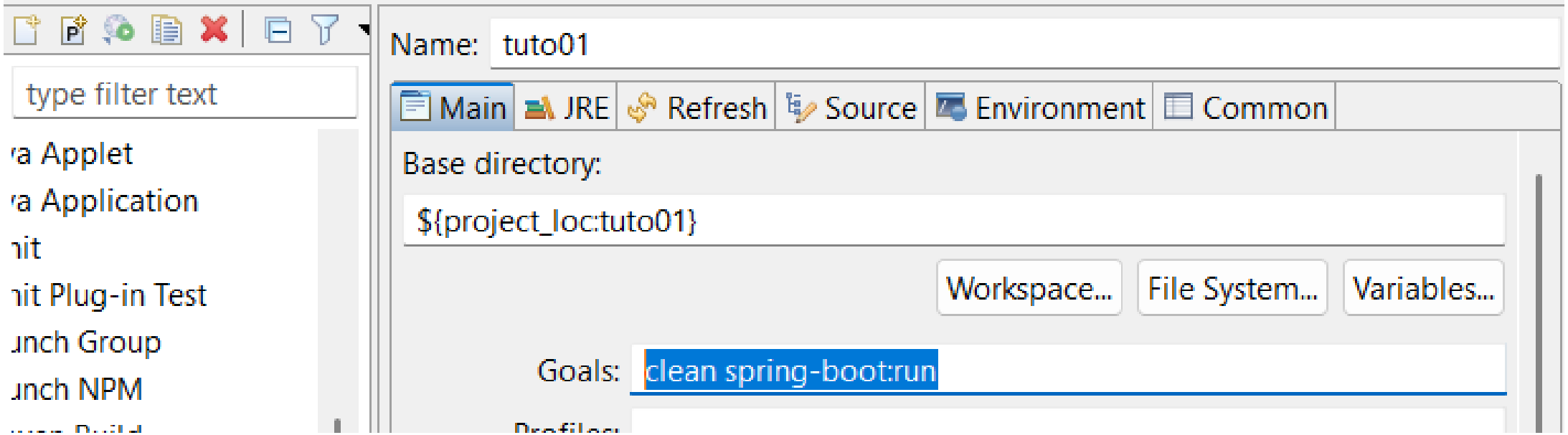
@RestController
public class ProductController {


}
```

Code example git repo

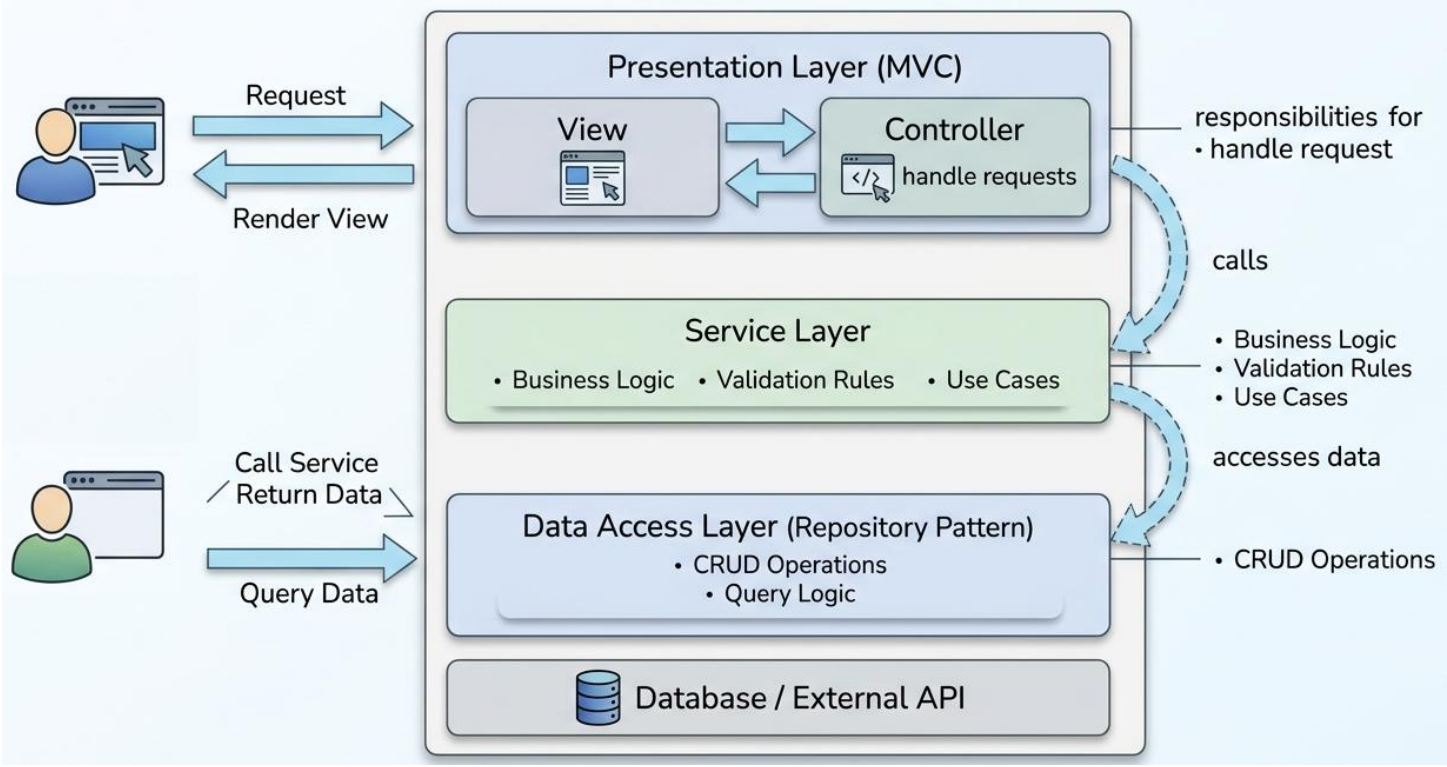
- All code examples in the next slides can be found on the git repository:

https://github.com/skanderturki/se411/tree/master/workspace_ch_06/tuto01



Architectural pattern: MVC + Service layer

Combine **MVC** with a **Service** Layer, by inserting the Service Layer as a mediator between the **Controller** and the Data Access Layer (the model, often a Repository pattern). This ensures that your controllers stay "thin" by delegating all business logic to the services.



- Define an interface for your service, Example: IUserService with methods like RegisterUser or GetUserProfile.
- Then, Implement the Service Class: Create a concrete class that implements the interface. This is where you put your business logic, such as validation rules, calculations, and coordinating multiple data operations.
- Instead of the Controller talking to the database, inject the service into the Controller's constructor.

Reference: SE371 chapter 07

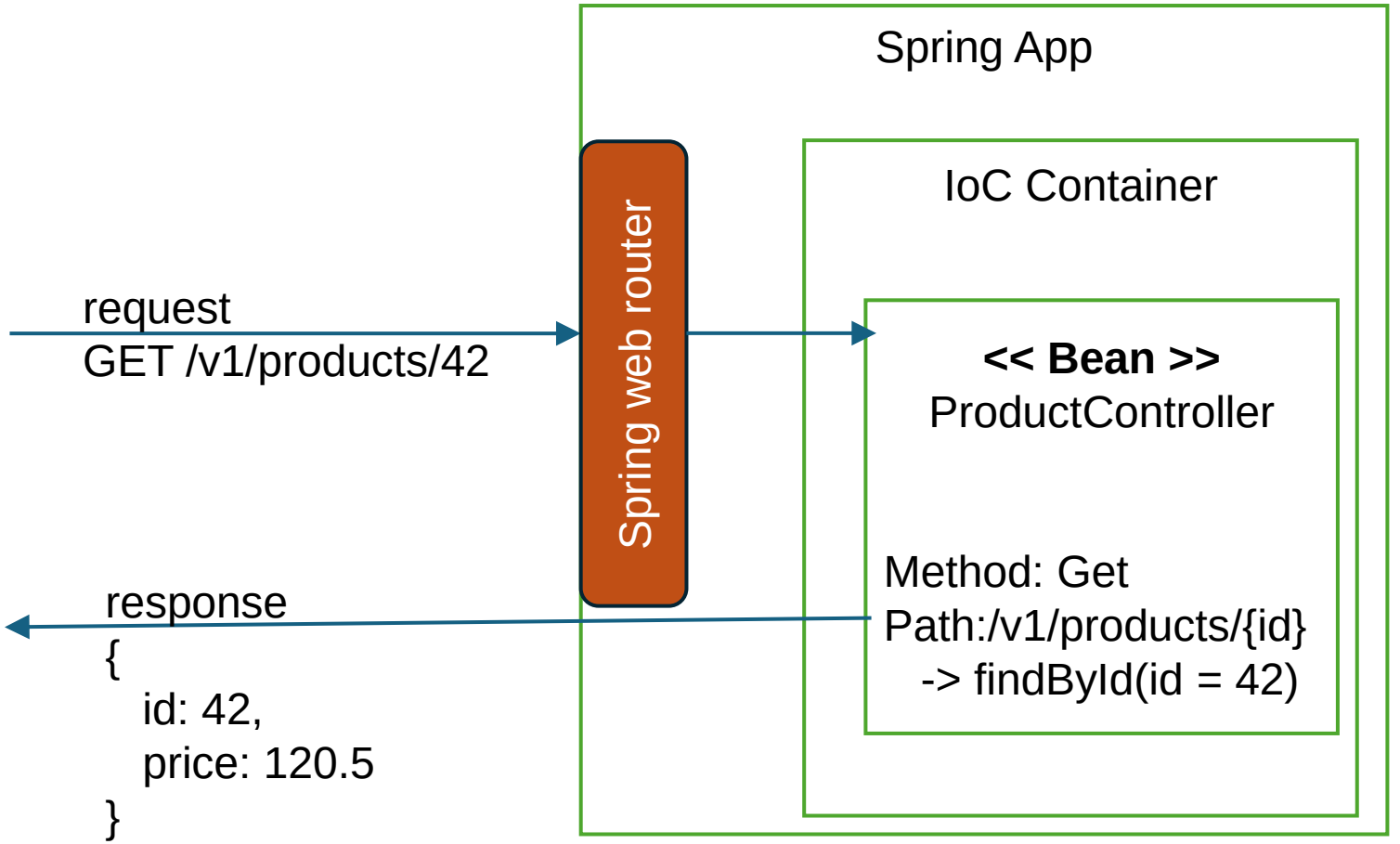
Spring Web Controllers

In Spring Web, Requests are handled by Controllers (***RestController):***

We create a Rest Controller by adding an annotation to the class: The Controller gets injected by Spring Web, which routes API requests (handled by the Controller) to the correct method.

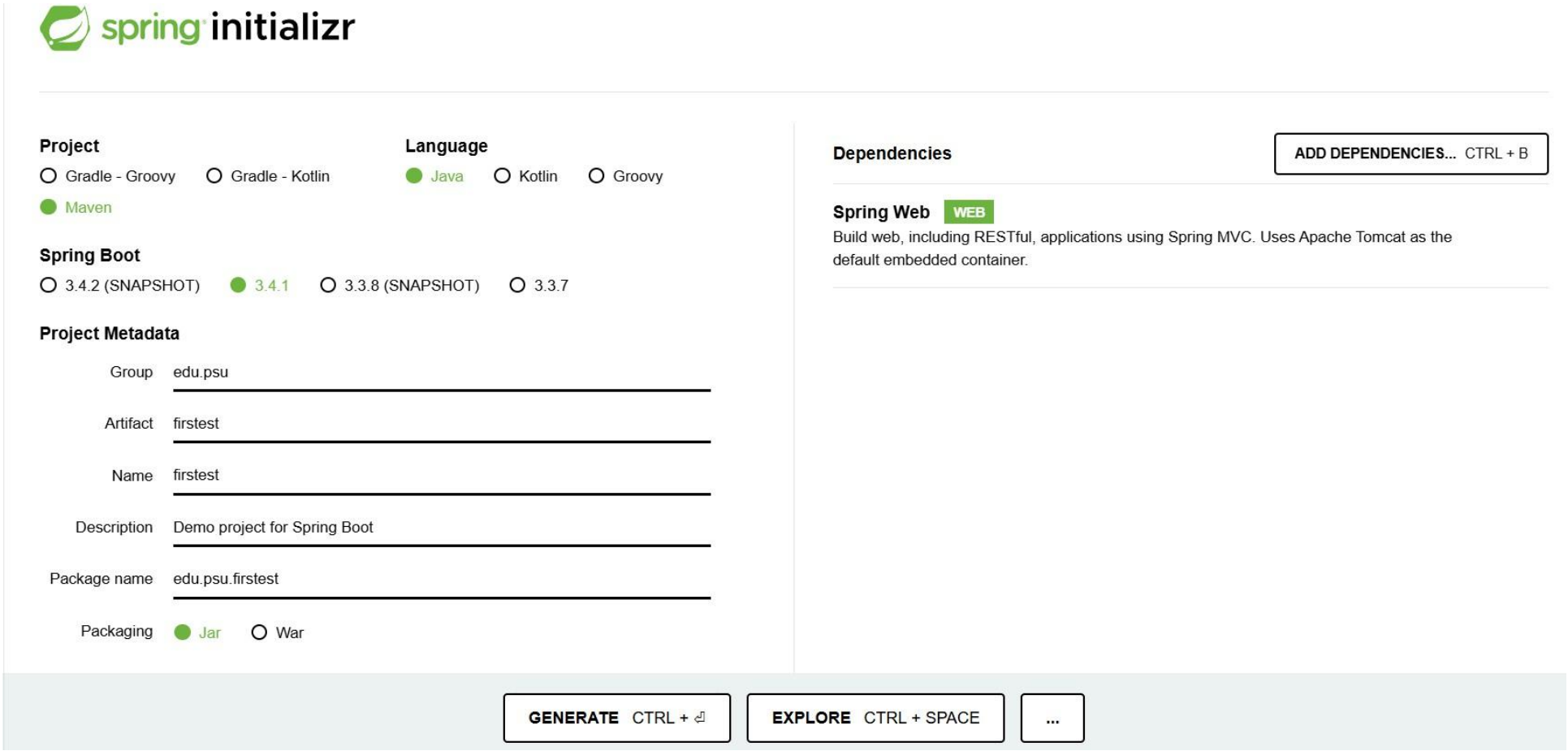
```
@RestController
class ProductController {

}
```



Spring Initializr

- Spring Initializr is an online service that is used to create a Spring boot application with the dependencies we need.



Controller handler methods

- We have to specify a method on the controller to handle requests (with a name of our choice). `example: GET /v1/products/23`
- We need to tell Spring to route requests to the method only on `GET` requests.
- We use `@GetMapping` annotation, which needs the URI Path.

```
@GetMapping("/v1/products/{requestedId}")

private Product findById(@PathVariable  Long requestedId) {

}
```

GET may return empty results

```
@GetMapping(“/v1/products/{requestedId}")
private ResponseEntity<Product> findById(@PathVariable Integer requestedId) {
  Optional<Product> productA = productRepository.findById(requestedId);
  if( productA.isPresent() ) {
    return ResponseEntity.ok(productA.get()) ;
  }
  else {
     return ResponseEntity.notFound().build();
   }
}
```

Controller handler methods

- Spring needs to know how to get the value of the `requestedId` parameter.
- This is done using the `@PathVariable` annotation.
- The fact that the parameter name matches the `{requestedId}` text within the `@GetMapping` parameter allows Spring to assign the correct value to the `requestedId` variable.

```
@GetMapping("/v1/products/{requestedId}")

private Product findById(@PathVariable  Long requestedId) {

}
```

Controller handler methods

- REST says that the Response needs to contain a Product in its body, and a Response code of 200 (OK).
- Spring Web provides the ResponseEntity class for this purpose.
- It also provides several utility methods to produce Response Entities.
- ResponseEntity represents the whole HTTP response: status code, headers, and body.
- Here, you can use ResponseEntity to create a response with code 200 (OK), and a body containing a Product.

Controller handler methods

- The final implementation looks like this.

```
@RestController class ProductController {

  @GetMapping("/products/{requestedId}")
  private ResponseEntity<Product> findById(@PathVariable Integer requestedId) {
    Optional<Product> product = productRepository.findById(requestedId);
    return ResponseEntity.ok(product.get());
  }
}
```

//This controller handler will respond to http://localhost:8080/products/5

The record keyword in JAVA

- In many cases, we create immutable data classes. These classes always look like the following example:

```
public class Person {
  private final String name;
  private final String address;

  public Person(String name, String address) { this.name = name; this.address = address; }

  @Override public int hashCode() { return Objects.hash(name, address); }

  @Override public boolean equals(Object obj) {   ...   }

  @Override public String toString() { return "Person [name=" + name + ", address=" + address + "]"; }
  // + standard getters
}
```

The record keyword in JAVA (2)

- Since Java 14, the **record** keyword allows us to create such classes easily:

```
public record Person (String name, String address) { }
```

- This creates the same immutable data class as in the previous slide.
- Then we can use it as a normal class:

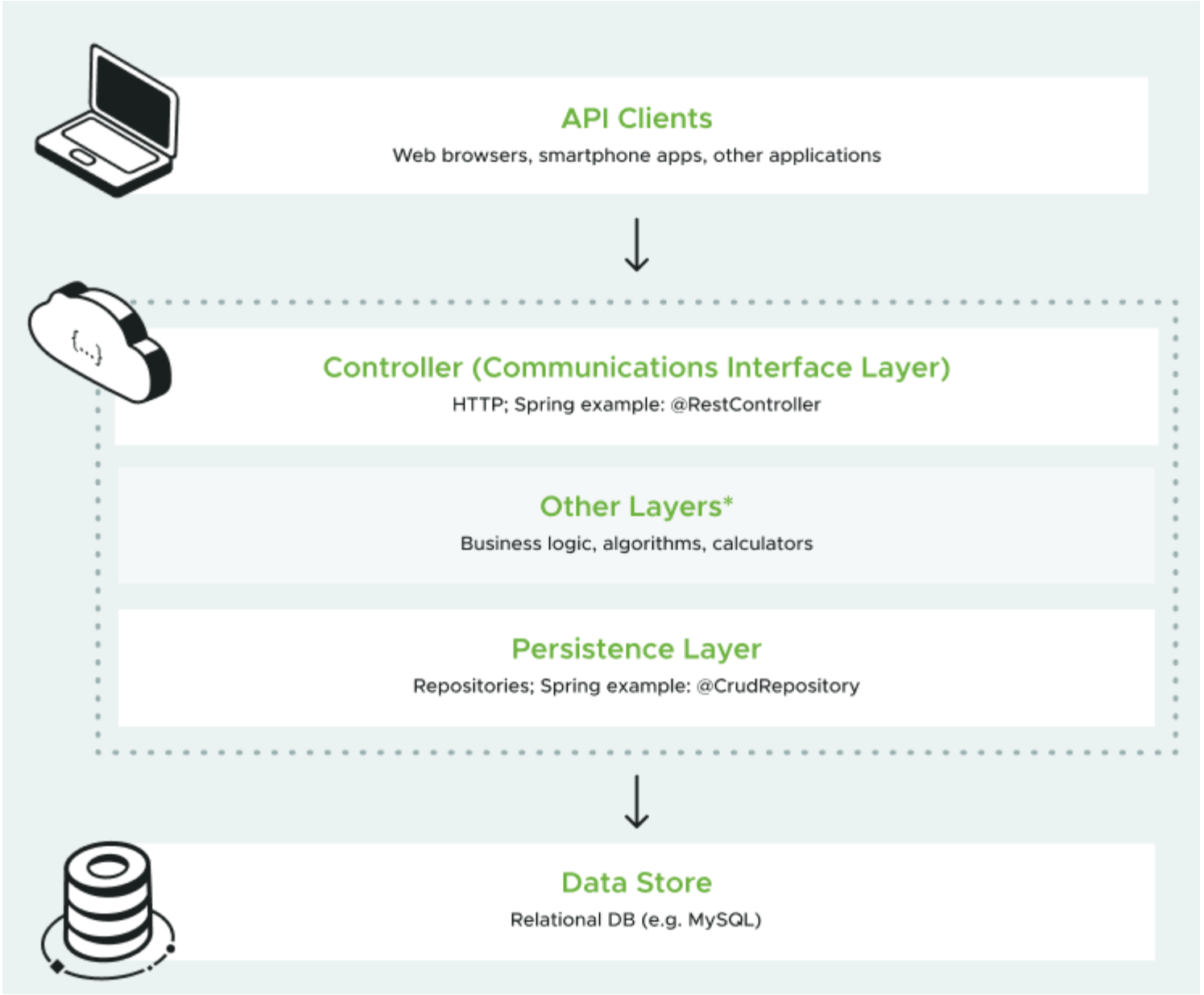
```
Person person = new Person("Salah", "48 Olaya street, Riyadh.");
```

The record keyword in JAVA

- Reduced Boilerplate: Records automatically generate constructors, getters, equals(), hashCode(), and toString() methods, minimizing the need for boilerplate code.
- Immutability by Design: Records are inherently immutable, which is crucial for maintaining data consistency and preventing unintended changes.
- Alignment with DTOs: Records are designed to be used as Data Transfer Objects (DTOs), which are typically immutable and used to hold data without additional behavior.

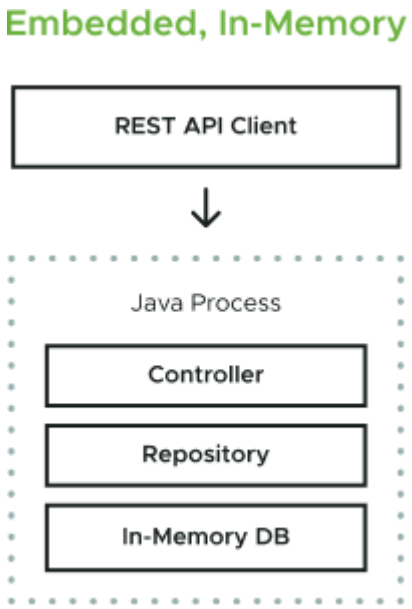
Repositories and Spring Data

- Spring Data works with Spring Boot to make database integration simple.
- The **Separation of Concerns** principle states that well-designed software should be modular, with each module having distinct and separate concerns from any other module (business, data, and presentation layers).



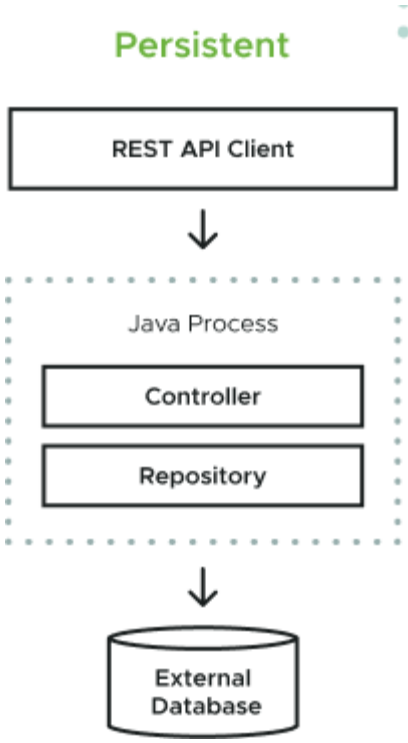
Database types

- **Embedded, in-memory** databases: “Embedded” means that it is a Java library, so it can be added to the project just like any other dependency. “In-memory” means that it stores data in memory only.



Database types

- **Persistent databases** are permanent, durable storage like relational database management systems (RDBMS) such as MySQL, SQL Server.
- We use JDBC (the standard Java library for database connectivity) and SQL (the standard database query language) with such databases.



Spring Data's CrudRepository

- For our Repository selection, we will use a specific type of Repository: Spring Data's CrudRepository.
- The following is a complete implementation of all CRUD operations by extending CrudRepository as below.

```
interface ProductRepository extends CrudRepository<Product, Integer> {}
```

- With just the above code, a caller can call any number of predefined CrudRepository methods, such as findById as below.

```
product = productRepository.findById(99);
```

Spring Data's CrudRepository (2)

Where is the implementation of
the ProductRepository.findById() method?

CrudRepository and everything it inherits from, is an Interface with no actual code. Based on the specific Spring Data framework used (which for us is Spring Data JDBC) Spring Data takes care of this implementation for us during the IoC container startup time. The Spring runtime will then expose the repository as another bean that you can reference wherever needed in your application.
The CrudRepository **generates SQL statements** to read and write your data, which is useful for many cases, but sometimes you need to write your own custom SQL statements for specific use cases.

How to link to the database using Spring

- Add the following dependencies:

```
<dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
</dependency>
```

activates Spring Data JDBC

adds the H2 database JAR to your application

- Automatically, Spring will create the embedded database, when starting the application we can see in the console:

HikariPool-1 - Added connection conn0: url=jdbc:**h2**:mem:56429977-72fc-4ecf-ac30-84a5a455f25a user=SA

Declaring the SQL Schema

- In Spring, if your database is not created yet, it can be defined using a schema.sql file in the resources folder (mostly useful for embedded databases):

schema.sql:

```
CREATE TABLE product
(
   ID int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   NAME varchar(100) NOT NULL,
   DESCRIPTION varchar(100),
   PRICE NUMBER
);
```

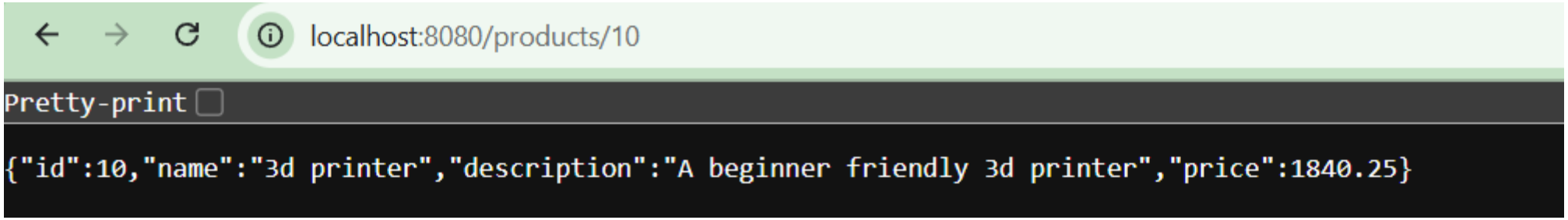
Creating testing data (mock data)

- By creating a file data.sql in the resources folder, Spring will execute the sql code to create the new mock data:

data.sql:

```
INSERT INTO PRODUCT(ID, NAME, DESCRIPTION, PRICE) VALUES (default, '3d printer',
'A beginner friendly 3d printer', 1840.25);
```

We can now send a get request to find a product by id:



Idempotent API as REST principle

- Idempotency means that the effect of a successfully performed request on a server resource is independent of the number of times it is executed (one or more).
- When we design the REST APIs, we must realize that API consumers can make mistakes.
- Consumers can write the client code in such a way that there can be duplicate requests coming to the APIs.
- These duplicate requests may be unintentional as well as intentional (e.g. due to timeout or network issues).
- We have to make our APIs fault-tolerant so that duplicate requests do not leave the system unstable.

Idempotent API as REST principle

- Also, idempotency is a key enabler for efficient caching and optimization strategies.
- Caches (like the Redis database) can store and serve the results of idempotent requests to reduce the load on servers and improve response times.
- If we follow the REST principles in designing our APIs, we will have automatically idempotent REST APIs for GET, PUT, DELETE. Only POST and PATCH are not idempotent.

HTTP PATCH is used for making partial updates to an existing resource without replacing the entire resource (like incrementing a value).

REST API endpoints: Implementing POST

These questions need to be answered before implementing our API endpoint:

1. Who specifies the ID - the controller, or the DB server?

   -> We can choose to let the DB server set the ID and manage the uniqueness of the ID, we can decide to set the ID from the controller component in situations where objects already have an ID like countries (966 for KSA).

2.Which HTTP method should we use in the Request? -> **POST**

3. What does the API send as a Response? : We could use 200 OK (the response that Read returns), but there's a more specific, more accurate code for resource creation with **POST**: **201** CREATED:

4.In Spring, the response can be created by provided methods:

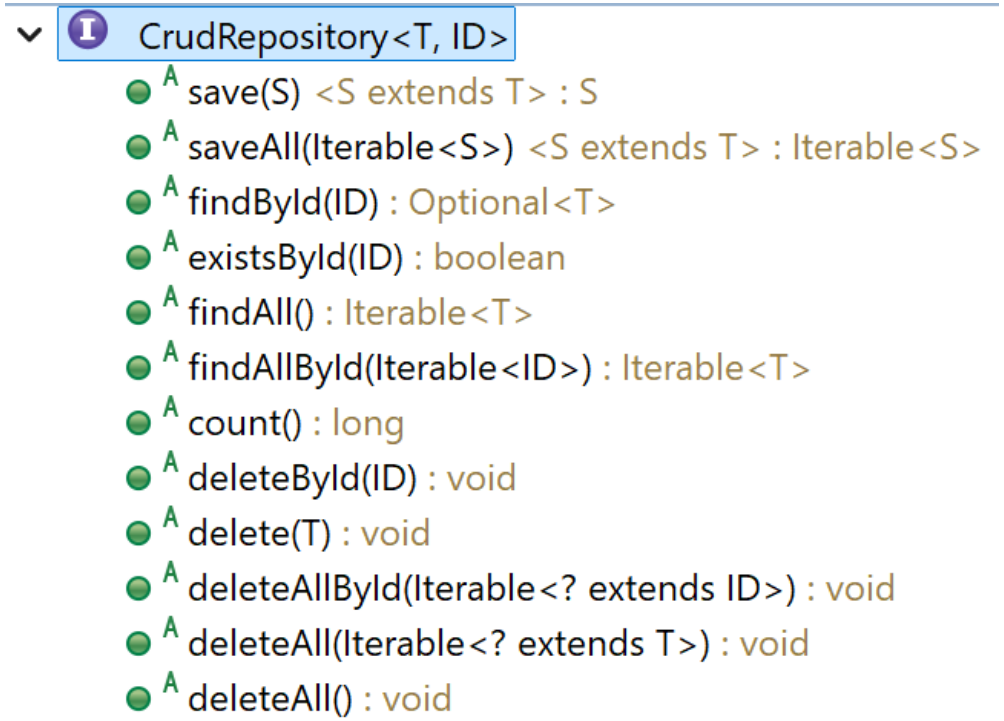
```
return ResponseEntity.created(…).build();
```

The POST method handler

- The POST method handler expects a request "body". This contains the data submitted to the API. Spring Web will deserialize the data into a Product object for us.

```
@PostMapping  // This allows spring to understand that this is a POST handler
private ResponseEntity<Void> createProduct(@RequestBody Product
newProductRequest, UriComponentsBuilder ucb) {
  Product newProduct = productRepository.save(newProductRequest);
  URI locationOfNewProduct = ucb.path("/v1/products/{id}")
          .buildAndExpand(newProduct.id())
          .toUri();
  return ResponseEntity.created(locationOfNewProduct).build();
}
```

Save the new item to the database



h

- This line in how we can insert the new item in the database with Spring:

```
Product newProduct = productRepository.save(newProductRequest);
```

- Spring Data's ProductRepository provides methods that support creating, reading, updating, and deleting data from a data store. productRepository.save(newProductRequest) saves a new Product and returns the saved object with a unique id provided by the database server.

The POST method handler

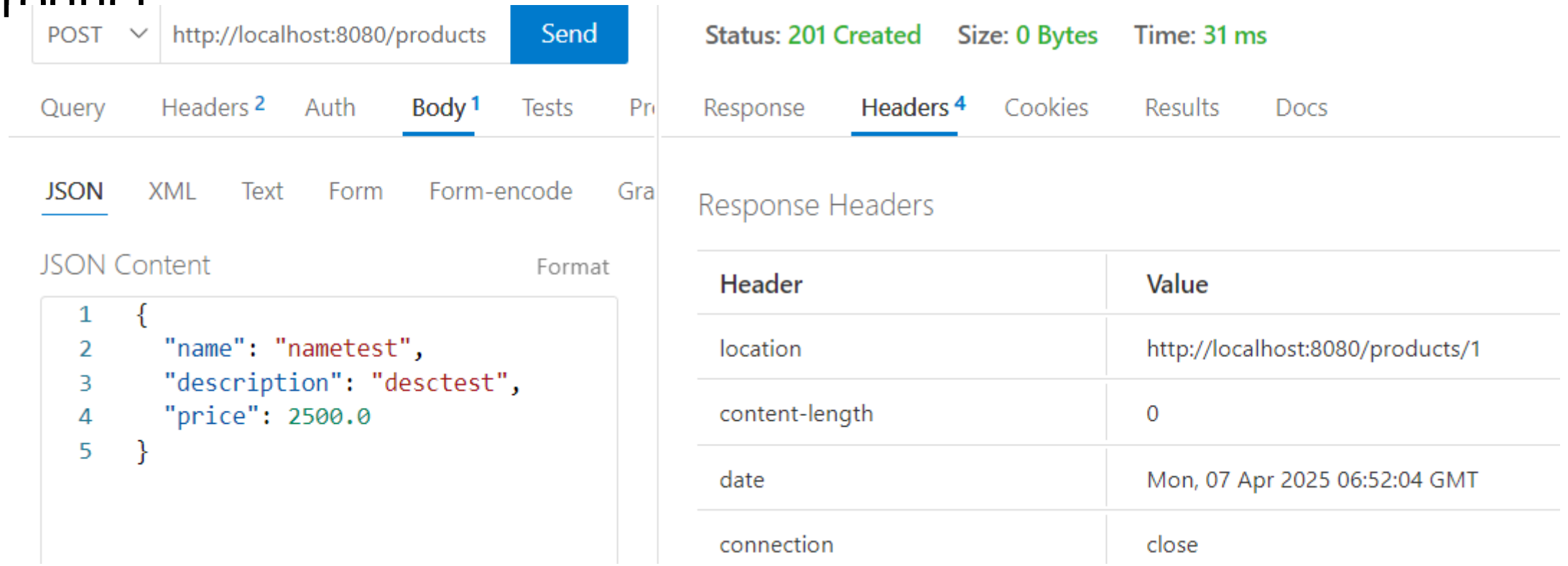
ucb.path… is constructing a URI to the newly created Product. This is the URI that the caller can then use to GET the newly-created Product (it is a requirement of HTTP to return the location of the newly created resource in the response, Ref: https://www.rfc-editor.org/rfc/rfc9110#name-201-created).

UriComponentsBuilder ucb is the second method argument to this POST handler method and is automatically passed and injected from the Spring's IoC Container.

```
@PostMapping  // This allows spring to understand that this is a POST handler
private ResponseEntity<Void> createProduct(@RequestBody Product
newProductRequest, UriComponentsBuilder ucb) {
  Product newProduct = productRepository.save(newProductRequest);
  URI locationOfNewProduct = ucb.path("/v1/products/{id}")
        .buildAndExpand(newProduct.id())
        .toUri();
  return ResponseEntity.created(locationOfNewProduct).build(); // returns 201
}
```

Testing the new POST handler

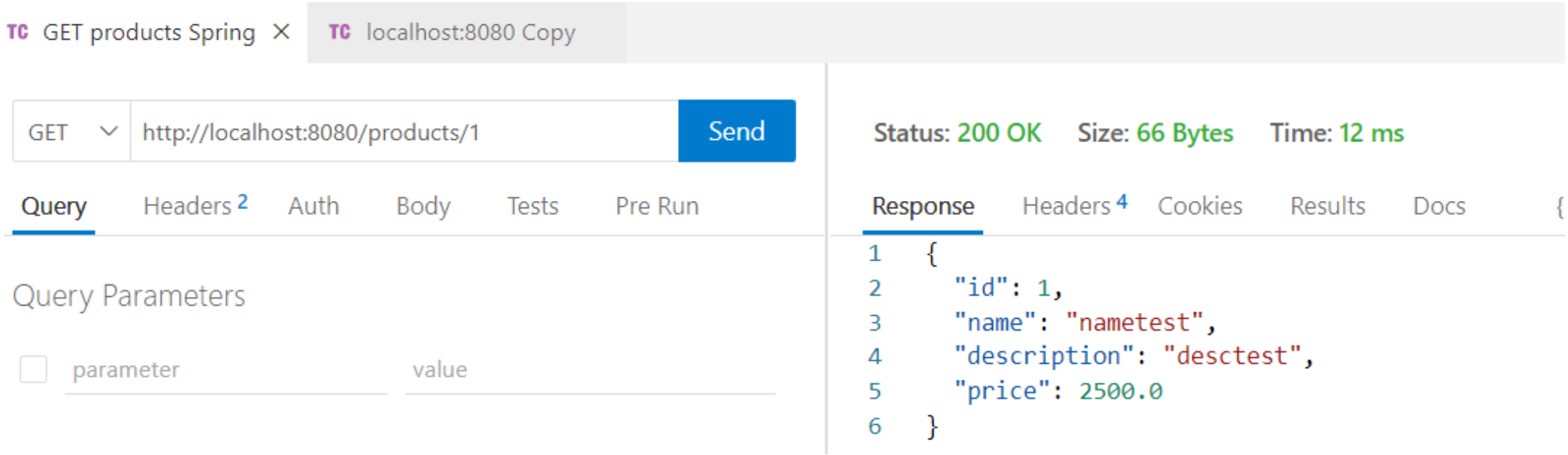
- We can use Postman or Thunder Client inside VScode to send a POST request with a body containing JSON data for the new Product



- In the response headers we received the location of the newly created product

Verify that the product was created

- We can see in the previous test that the new product has the id of 1, so we can send a GET request to check that the data was correctly created:



Try POST by providing an id

JSON Content

```
1  {
2    "id": 20,
3    "name": "nametest",
4    "description": "desctest",
5    "price": 2500.0
6  }
```

- If we try to provide an id we would get this error:
- `org.springframework.dao.IncorrectUpdateSemanticsDataAccessException: Failed to update entity [Product[id=20, name=nametest, description=desctest, price=2500.0]]; Id [20] not found in database`

- The Repository is trying to find Product with id of 20 and throwing an error when it cannot find it. Supplying an id to productRepository.save() is supported when an update is performed on an existing resource.

Try POST by providing an id (2): Solution

- To improve our REST API robustness, we include the “save” method inside a try/catch block:

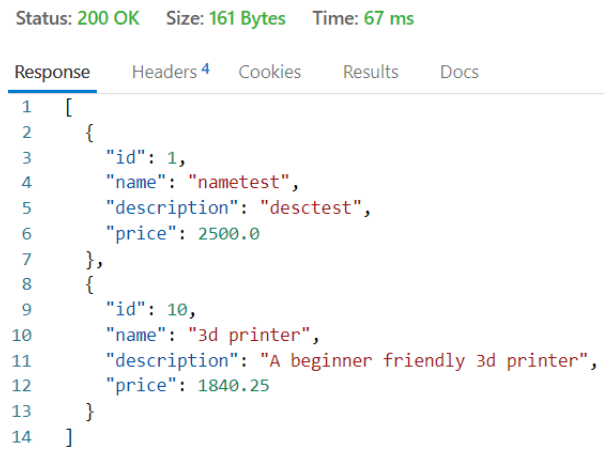
```
@PostMapping
public ResponseEntity<Void> createProduct(@RequestBody Product newProductRequest
                                          , UriComponentsBuilder ucb) {
   try {                    // Catch the errors caused by “save”
      Product newProduct = productRepository.save(newProductRequest);
      URI locationOfNewProduct = ucb.path("/v1/products/{id}")
                                    .buildAndExpand(newProduct.id()).toUri();
      return ResponseEntity.created(locationOfNewProduct).build();
   } catch (Exception ex) {
      return ResponseEntity.unprocessableEntity().build();   // return 422 code
   }
}
```

UnprocessableEntity 422 code is returned if the “save” method fails, usually because the database server is expecting a different format of the data

Requesting the list of all Products (GET)

- `ProductRepository` has a `findAll` method that we can use to easily fetch all the Products from the database:

```
@GetMapping()
public ResponseEntity<Iterable<Product>> findAll() {
    try {
        Iterable<Product> products = productRepository.findAll();
        return ResponseEntity.ok(products) ;
    } catch (Exception ex) {
        return ResponseEntity.internalServerError().build() ;
    }
}
```



Pagination and sorting

- To implement pagination and sorting, we'll use a specialized version of the CrudRepository, called the PagingAndSortingRepository.
- An API should **not** be able to produce a response with unlimited size, because this could overwhelm the client or server memory, not to mention taking quite a long time.
- Pagination in Spring (and many other frameworks) works by specifying the page length (e.g. 10 items) and the page index (starting with 0).
- In order for pagination to produce the correct page content, the items must be sorted in some specific order.

Spring Data Pagination API

- Spring Data provides the `PageRequest` and `Sort` classes for pagination:

```
Page<Product> page2 = productRepository.findAll(
      PageRequest.of(        // simply returns a PageRequest object
          1,           // page index for the second page - indexing starts at 0
          10,        // page size (the last page might have fewer items)
          Sort.by(new Sort.Order(Sort.Direction.DESC, “price"))
    )
);
```

Spring Data Pagination: The URI

- We have to design the URI of the new endpoint:
- Examples:
  - Get the second page: /v1/products?page=1
  - Page with length of 10: /v1/products?page=1&size=10
  - ..sorted by price: /v1/products?page=1&size=10&sort=price
  - ..in descending order: /v1/products?page=1&size=10&sort=price,desc

The pagination handler

- Add PagingAndSortingRepository to the ProductRepository:

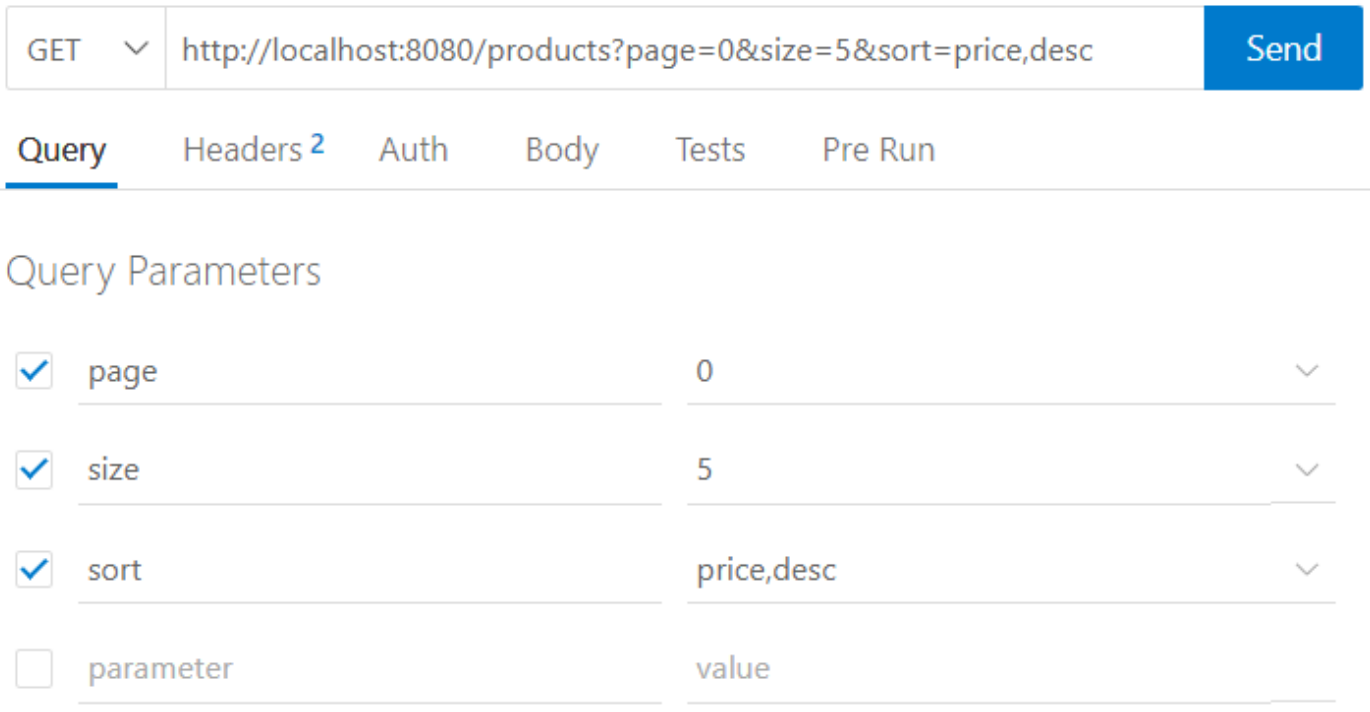
```
public interface ProductRepository extends CrudRepository<Product, Integer>,
                                           PagingAndSortingRepository<Product, Integer>{ }
```

- Implement the handler:

```
@GetMapping
public ResponseEntity<List<Product>> findAll(Pageable pageable) {
   try {
      if(pageable.getPageSize() > 100) { // Response size limiting (default is 20)
         return ResponseEntity.badRequest().build();
      }
      Page<Product> page = productRepository.findAll(PageRequest.of(pageable.getPageNumber(),
               pageable.getPageSize(), pageable.getSortOr(Sort.by(Sort.Direction.DESC, "price"))));
      return ResponseEntity.ok(page.getContent());
   } catch (Exception ex) {
      return ResponseEntity.internalServerError().build();
   }
}
```

Testing the paginated findAl

http://localhost:8080/products?page=0&size=5&sort=price,desc



Status: 200 OK Size: 454 Bytes Time: 8 ms

Response Headers 4 Cookies Results Docs


```
[
  {
    "id": 18,
    "name": "3d printer9",
    "description": "A beginner friendly 3d printer",
    "price": 3640.25
  },
  {
    "id": 16,
    "name": "3d printer7",
    "description": "A beginner friendly 3d printer",
    "price": 3240.25
  },
  {
    "id": 15,
    "name": "3d printer6",
    "description": "A beginner friendly 3d printer",
    "price": 2840.25
  },
  {
    "id": 17,
    "name": "3d printer8",
    "description": "A beginner friendly 3d printer",
    "price": 2840.25
  },
  {
    "id": 26,
    "name": "PC 08",
    "description": "A portable computer",
    "price": 2440.25
  }
```

Implementing PUT (update)

h

To update an existing database record we need:

- Create a new endpoint to receive an HTTP request with:
  - the verb PUT, (can use PATCH for partial updates such as incrementing a value or updating a salary or a date)
  - The same URI as for the “create” operation, (+ the ID)
  - and a JSON object in the body
- Return an appropriate response from the endpoint for:
  - success (204 no content)
  - and error conditions (404 not found, 500 internal server error)

Response Body and Status Code

For an update, we need to return 204 NO CONTENT, and an empty response body.

The rationale in this case is that since a PUT simply places an object at the URI in the request, the client doesn't need any information back - it knows that the object in the request has been saved, on the server.

We also need to return 404 NOT FOUND for non-existent IDs, and 500 internal server error in case of any unknown server-side problem (such as DB connection failure).

Implementing PUT (update) (2)

The endpoint mapping:

```
@PutMapping("/{requestedId}")
private ResponseEntity<Void> updateProduct(@PathVariable Integer requestedId,
                                           @RequestBody Product productUpdate) {

…..
}
```

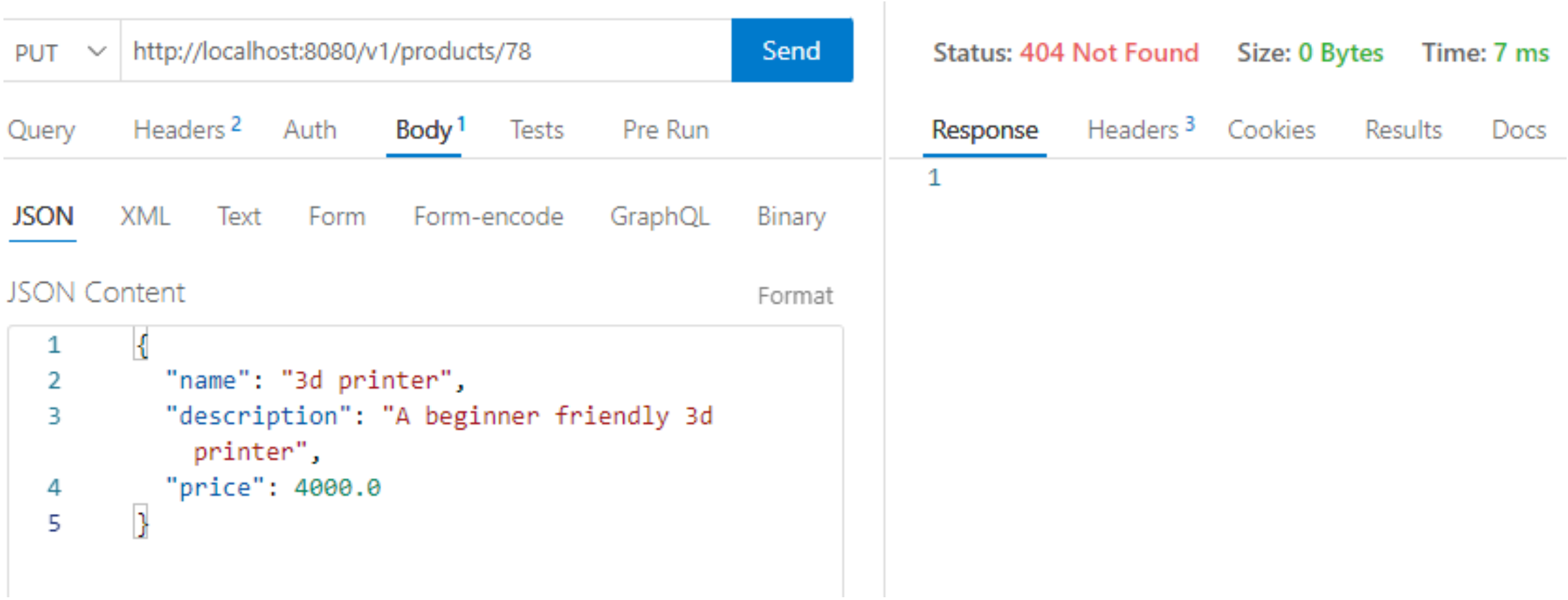
Source code: https://github.com/skanderturki/se411/tree/master/workspace_ch_06/crud02

Implementing PUT (update) (3)

```
@PutMapping("/{requestedId}")
public ResponseEntity<Void> createProduct(@PathVariable Integer requestedId,
                                          @RequestBody Product productUpdate) {
  try {
    Optional<Product> product = productRepository.findById(requestedId); // find the record
    if (product.isPresent()) {
      Product updatedProduct = new Product(product.get().id(), productUpdate.name(), // new object
      productUpdate.description(), productUpdate.price()); // with the same id

      productRepository.save(updatedProduct); // save the new object
    } else {
      return ResponseEntity.notFound().build(); // Object with id not found
    }
    return ResponseEntity.noContent().build(); // Update Success
  } catch (Exception ex) {
    return ResponseEntity.internalServerError().build();
  }
}
```

Trying to update a non-existing record



Updating an existing record

PUT http://localhost:8080/v1/products/18 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1   {
2     "name": "3d printer",
3     "description": "A beginner friendly 3d
        printer",
4     "price": 4000.0
5   }
```

Status: 204 No Content Size: 0 Bytes Time: 8 ms

Response Headers 2 Cookies Results Docs

1

Implementing DELETE

Here is he specification that we'll define for the DELETE:

Request:
- Verb: `DELETE`
- URI: `/v1/products/{id}`
- Body: *(empty)*

Response:
- Status code: `204 NO CONTENT` or `404 NOT FOUND`
- Body: *(empty)*

We'll return the
`204 NO CONTENT` status code for a successful delete
`404 NOT FOUND` if the record is not found

Hard and Soft Delete

We need to decide what happens to resources when they are deleted.

Hard delete is to delete the record from the database. With a hard delete, it's gone forever. -> *What can we do if we need data that existed prior to its deletion?*

An alternative is **Soft delete** which works by marking records as "deleted" in the database (so that they're retained, but marked as deleted).

For example, we can introduce an IS_**DELETED** boolean or a DELETED_DATE timestamp column and then set that value-instead of fully removing the record by deleting the database row(s).

With a soft delete, we also need to change how Repositories interact with the database. For example, a repository needs to respect the "deleted" column and exclude records marked deleted from Read requests.

Audit Trail and Archiving

When working with databases, you'll find that there's often a requirement to keep a record of modifications to data records. For example:

A customer service representative might need to know when a customer deleted their Data.

There may be ***data retention compliance*** regulations which require deleted data to be retained for a limited period of time.

If the record is hard-deleted then we'd need to store additional data to be able to trace previous state. Let's discuss some ways to record historical information:

- Archive (move) the deleted data into a different location.
- Add audit fields to the record itself. For example, a DELETED_DATE column. Additional audit fields can be added, for example DELETED_BY_USER.
- Maintain an audit trail. The audit trail is a record of all important operations done to a record. It can contain not only Delete operations, but Create and Update as well.

Audit Trail and Archiving (2)

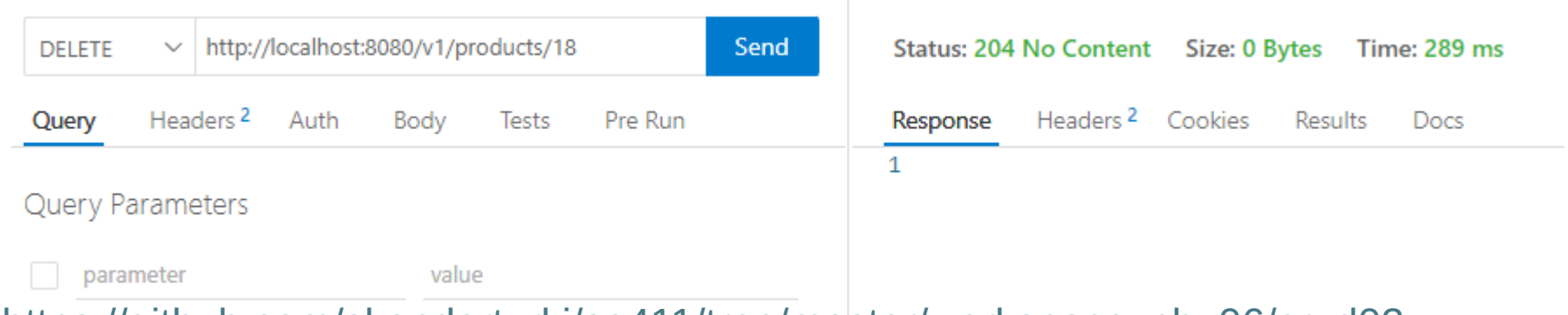
The advantage of an audit trail over audit fields is that a trail records all events, whereas audit fields on the record capture only the most recent operation. An audit trail can be stored in a different database location, or even in log files.

Here are some examples of strategies for implementing “delete”:

- We could implement soft delete, then have a separate process which hard-deletes or archives soft-deleted records after a certain time period, like once per year.
- We could implement hard delete, and archive the deleted records.
- In any of the above cases, we could keep an audit log of which operations happened and when.

Implementing DELETE (2)

```
@DeleteMapping("/{requestedId}")
public ResponseEntity<Void> deleteProduct(@PathVariable Integer requestedId) {
  try {
     Optional<Product> product = productRepository.findById(requestedId);
     if (product.isPresent()) {
       productRepository.delete(product.get());
     } else {
       return ResponseEntity.notFound().build();
     }
     return ResponseEntity.noContent().build();
  } catch (Exception ex) {
    return ResponseEntity.internalServerError().build();
  }
}
```

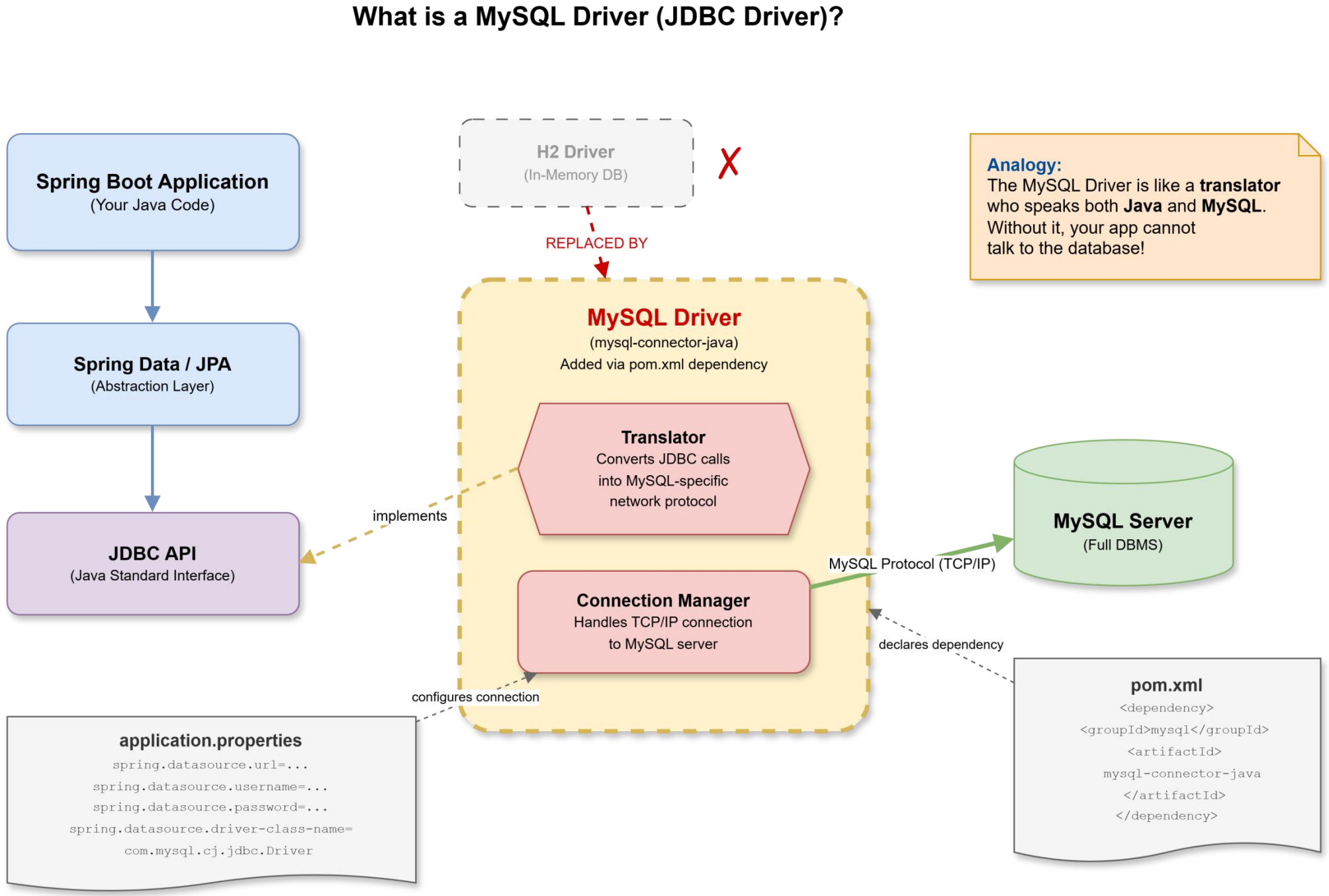


Source code: https://github.com/skanderturki/se411/tree/master/workspace_ch_06/crud02

Using a full-fledged DBMS (MySQL)

- After testing our server using an in-memory database (h2), we can now switch to a full-fledged DBMS. We will use MySQL. We need to make three changes:
  - make slight modifications to the SQL scripts as MySQL use a slightly different **dialect** of the SQL language.
  - Change the imported dependencies in the pom.xml to use **MySQL driver** instead of the H2 driver (API).
  - Add the connection configuration to allow Spring Data to find our server and have the necessary connection details.

MySQL Driver or connector



MySQL’s SQL Dialect and case sensitivity

- In MySQL, databases correspond to directories within the data directory. Each table within a database corresponds to at least one file within the database directory (and possibly more, depending on the storage engine). Triggers also correspond to files. Consequently, the case sensitivity of the underlying operating system plays a part in the case sensitivity of database, table, and trigger names.
- This means such names are not case-sensitive in Windows, but are case-sensitive in most varieties of Unix.
- In Spring, table names through the Repository classes use lower case characters (product, employee, etc).
- So, we need to use lower case letters for database, table and trigger names in our SQL queries.

MySQL versions of previous SQL queries

H2	MySQL

`// no need to drop table when server`
`// starts as nothing is persisted (in-memory)`
 	
`CREATE TABLE product`
`(`
`ID int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,`
`NAME varchar(100) NOT NULL,`
`DESCRIPTION varchar(100),`
`PRICE NUMBER`
`);`
	`DROP TABLE IF EXISTS product;`
	
	`create table product`
	`(`
	`id int not null auto-increment PRIMARY KEY,`
	`name varchar(100) not null,`
	`description varchar(100),`
	`price DECIMAL(8,2)`
	`);`

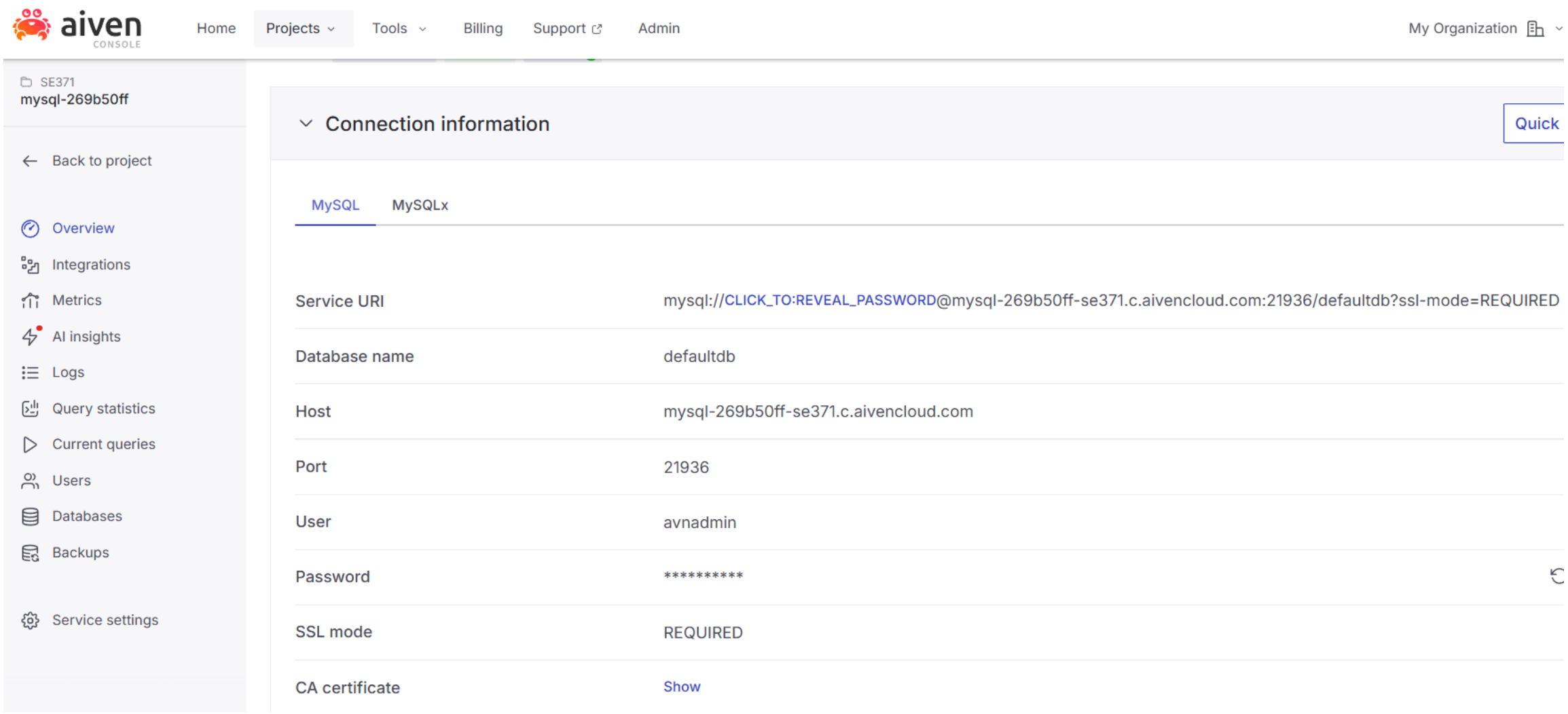
`INSERT INTO PRODUCT(ID, NAME, DESCRIPTION, PRICE) VALUES (10, '3d printer', 'A beginner friendly 3d printer', 1840.25);`	`insert into product(name, description, price) values ('3d printer', 'A beginner friendly 3d printer', 1840.25);`

- In MySQL, we can use “auto-increment” for the id, so we do not need to set the id in the INSERT INTO statement

Change the dependencies in pom.xml

H2	MySQL
<dependency> <groupId>org.springframework.data</groupId> <artifactId>spring-boot-starter-data-jdbc</artifactId> </dependency>	<dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-data-jdbc</artifactId> </dependency>
<dependency> <groupId>com.h2database</groupId> <artifactId>h2</artifactId> </dependency>	<dependency> <groupId>mysql</groupId> <artifactId>mysql-connector-java</artifactId> <version>8.0.33</version> </dependency>

Using Aiven as a cloud MySQL DB server



Configure DB connection details

- In application.properties, we need to specify the properties of the database connection.

```
spring.datasource.url=jdbc:mysql://mysql-269b50ff-se371.c.aivencloud.com:21936/se411_products
spring.datasource.username=avnadmin
spring.datasource.password=AVNS_dfUKLOn_qg5hlzfJ309
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.sql.init.mode=always
```

Code example git repo

- All code examples in the slides can be found on the git repository.

  https://github.com/skanderturki/se411/tree/master/workspace_ch_06/

END